



## DPUArduinoNetIO

Communicates with an Arduino-based microcontroller over the network and controls its analog and digital terminals. The Arduino must run the corresponding “NetIO” sketch (see “Functional principle”).

### 1. Dynamic parameters:

*Sys\_SaveStartup*

Save the current configuration as a startup configuration (see the corresponding section below). The startup configuration is restored every time the Arduino is switched on.

### 2. Static parameters:

*Sys\_IP*

IP address of the Arduino controller.

*Sys\_Port*

UDP port set in the Arduino controller.

*Default: 8888*

*Sys\_TimeoutMS*

Watchdog timeout [ms]. When SILAB sends no data for this timespan (for example, because SILAB has crashed or the network connection is broken), the Arduino sets all output pins to their default values.

*Default: 1000*

### 3. Outputs:

*Sys\_NPacketsSent*

Counts the number of UDP packets sent by SILAB to the Arduino.

*Sys\_NPacketsReceived*

Counts the number of UDP packets received by SILAB from the Arduino. Under normal circumstances, these numbers should not differ by more than 2%.

### 4. Functional principle:

DPUArduinoNetIO communicates with an Arduino-based microcontroller over the network. The following Arduino boards are currently supported:

- Arduino Uno with Ethernet Shield or Ethernet Shield 2
- Arduino Ethernet with USB Serial adapter
- Arduino Mega 2560 with Ethernet Shield or Ethernet Shield 2

Please note that the Ethernet Shield and the Arduino Ethernet are no longer sold. To use the replacement Ethernet Shield 2, Arduino IDE 1.8 or newer and NetIO sketch version 2018 or newer are required.

Each of the Arduino boards has a different number of input and output pins:



# SILAB DRIVING SIMULATION

version Version 7.1 documentation

- The Arduino Uno and the Arduino Ethernet have
  - six analog input pins, labeled A0 ... A5
  - fourteen digital I/O pins, labeled D0 ... D13.
    - Pins D0, D1, D4 and D10...D13 are used by on-board hardware and cannot be used by DPUArduinoNetIO.
    - Analog output (PWM) is only possible for pins D3, D5, D6 and D9.
- The Arduino Mega has
  - sixteen analog input pins, labeled A0 ... A15
  - fifty-four digital I/O pins, labeled D0 ... D53.
    - Pins D0, D1, D4, D10 and D50...D53 are used by on-board hardware and cannot be used by DPUArduinoNetIO.
    - Analog output (PWM) is possible on pins D2...D13 and D44...D46.

To set up the Arduino controller, the following steps must be performed:

- Load the NetIO sketch onto the controller using the USB connection.
- Configure the controller's network settings over the USB connection.
  - The serial port emulation must be set to 19200 baud.
  - NetIO provides a "console" which allows network configuration.
    - The h command provides help
    - The l command lists the current settings
    - The commands m, i, s and g are used to configure network settings. It is especially important to assign a unique MAC and IP address to the controller.
  - The network settings are automatically stored in the controller's non-volatile storage ("EEPROM") and restored whenever it is powered on.
- Restart the controller and connect it to the LAN. The USB connection is now no longer required. The controller can instead be powered over the power input socket (+7V ... +12V).

## 5. Additional inputs and outputs:

Every input and output pin of the Arduino board that is not already used by an on-board component can be configured and a corresponding SILAB variable can be defined.

## 6. Analog inputs:

To use the Arduino's analog input pins (A0..An), use the following syntax:

```
Ai = ("Name", <Default>);
```



This creates a DPU output variable called "Name", whose value is <Default> while the simulation is not running. As soon as the simulation is running, the current volt-age on the analog input is measured. The output range is 0 = GND up to 1 = +5V.

An extended syntax is available to assign a different range to the SILAB variable:

```
Ai = ("Name", <Default>, <Multiplier>, <Offset>);
```

The value of the SILAB variable is then <Offset> = GND up to <Offset>+<Multiplier> = +5V.

Please note that negative values of <Multiplier> can be used to "invert" the variable.

## 7. Digital I/O:

To use the Arduino's digital I/O pins (D0..Dn), use the following syntax:

```
Di = ("Name", <Direction>, <Mode>, <Default>);
```

This creates a DPU variable called "Name", whose value is <Default> while the simulation is not running.

<Direction> is **Read** for a digital input pin, and **Write** for a digital output pin.

Each digital input pin is represented by a DPU output variable called "Name" and can be configured using the <Mode> parameter:

- <Mode> = **Pullup** enables the Arduino's internal 20 kΩ pull-up resistor and is recommended for switches.
- <Mode> = **Open** disables the pull-up resistor.

Each digital output pin is represented by a DPU input variable called "Name" and can be configured using the <Mode> parameter:

- <Mode> = **Digital** uses digital output. Whenever "Name" is set to a value greater than 0.5, the output pin is set to +5V; otherwise it is set to GND.
- <Mode> = **Analog** (only on supported pins) enables the Arduino's internal PWM generator for this pin (which uses a base frequency of ~ 600 Hz) and modifies its duty cycle according to the value of the SILAB variable: For the value 0, the duty cycle is 0%; and it rises up to 100% for the value 1. The out-put voltage is always +5V.

An extended syntax is available to assign a different range to the SILAB variable:

```
Di = ("Name", <Direction>, <Mode>, <Default>, <Multiplier>, <Offset>);
```

The meaning of these parameters depends on the <Direction> and <Mode> settings.

For **Analog Write** variables, the SILAB variable's range is changed to <Offset> = 0% duty up to <Offset>+<Multiplier> = 100% duty.

Please note that negative values of <Multiplier> can be used to "invert" the variable.



For **Digital Write** and **Digital Read** variables, the only supported option is to negate the variable (the meanings of 0 and 1 are reversed). To do so, specify *<Multiplier>* = **Invert**. The *<Offset>* parameter is not required in this case and can be omitted.

## 8. Startup configuration:

It is possible to store a I/O configuration, and its default output values, in the Arduino's non-volatile storage ("EEPROM"). To do so, change the DPU input *Sys\_SaveStartup* from 0 to non-zero during the simulation. Please check SILAB's message window, it will show if the operation succeeded.

When the Arduino is powered on, it checks for a valid I/O configuration in its EEPROM. If one is found, it is restored immediately and the output pins are set to the values that were specified as *<Default>* parameters of the SILAB configuration.

## 9. Dependencies:

The Arduino board must be reachable on the local network (under the address as specified in the static parameter *Sys\_IP*). The simulation cannot be started unless the Arduino is switched on and connected.

There are no dependencies to other DPUs.

## 10. Example:

This DPU works well using the following script:

```
DPUArduinoNetIO IO
{
    Computer = {LOCALHOST};
    Index = 1;

    # Connect to the Arduino at this IP address and port:
    Sys_IP = "10.1.1.68";
    Sys_Port = 8888;

    # Enable two analog inputs for a joystick controller
    # GND is mapped to -1 in SILAB; +5V is mapped to +1.
    # This is the same convention as used by DPUDX8Joystick.
    # The default values are 0, corresponding to the middle position.
    A0 = (JoyX, 0, 2, -1);
    A1 = (JoyY, 0, 2, -1);

    # Control two LEDs, the first one using a PWM signal
    # the second one is only switched on or off (but is on by default).
    D3 = (PWMLight, Write, Analog, 0.05);
    D5 = (Light, Write, Digital, 1);
}
```



## SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
# Read two switches using the Arduino's internal pull-up
# resistors. The switch pulls down to GND when pressed;
# this is mapped to the SILAB output 1 using the "Invert"
# multiplier. When released, the SILAB output is 0.
D8 = (Taste1, Read, Pullup, 0, Inv);
D9 = (Taste2, Read, Pullup, 0, Inv);
};
```